

Sim-to-Real Transfer in Robot Learning: A Reinforcement Learning Approach

Zahra Sadeghi Jalalabadi
Politecnico di Torino
s329869

Abstract—This project explores reinforcement learning (RL) for robot control, focusing on the sim-to-real transfer challenge. Using the Hopper environment in MuJoCo, we train policies with SAC algorithms and evaluate their generalization using Uniform Domain Randomization (UDR). Results indicate that UDR improves policy robustness, reducing the performance gap between simulation and real-world conditions. As extension to project, The ADR is implemented.

I. INTRODUCTION

Reinforcement learning (RL) has shown great potential in robotics, but transferring policies trained in simulation to real-world environments remains challenging. This project investigates sim-to-real transfer using the Hopper robot in MuJoCo, implementing domain randomization (UDR,ADR) to improve generalization.

II. RELATED WORK

Several studies address the sim-to-real transfer problem. Sutton and Barto [1] provide foundational RL concepts. Tobin et al. [2] introduce domain randomization, while Peng et al. [3] demonstrate its effectiveness in robotic control.

III. METHODOLOGY

A. Hopper Environment

The hopper is a two-dimensional one-legged figure that consist of four main body parts - the torso at the top, the thigh in the middle, the leg in the bottom, and a single foot on which the entire body rests. The goal is to make hops that move in the forward (right) direction by applying torques on the three hinges connecting the four body parts.

1) *Reward Structure*: The reward function incentivizes efficient and controlled hopping, consisting of:

- **Healthy Reward**: Fixed reward for staying upright.
- **Forward Reward**: Encourages forward movement.
- **Control Cost**: Penalizes excessive torques to encourage smooth control.

The total reward function is:

$$\text{reward} = \text{healthy_reward} + \text{forward_reward} - \text{ctrl_cost} \quad (1)$$

2) *Episode Termination Conditions*: An episode ends when:

- The **hopper becomes unstable**, meaning:
 - It **falls** (height out of bounds).
 - The **torso angle** exceeds a safe range.

- **Maximum episode length** (1000 timesteps) is reached.
- If `terminate_when_unhealthy=True`, the episode ends immediately upon instability; otherwise, it runs until the time limit.

In this project, the Hopper environment consists of two main domains target and source. I have introduced other domains for my work UDR and ADR.

- **Source domain**: Includes a 1kg offset in torso mass.
- **Target domain**: Represents real-world conditions without offset.
- **UDR domain**: Includes a 1kg offset in torso mass and the uniform randomized weight on other body parts.
- **ADR domain**: Includes a 1kg offset in torso mass and automatically randomizing weights on other body parts based on the performance of agent.

B. Reinforcement Learning Pipeline

Reinforcement Learning (RL) is a type of Machine Learning, but unlike Supervised Learning, where the dataset is static and a gradient descent algorithm is used to achieve convergence, RL generates data dynamically. In RL, the agent interacts with the environment, and the learning policy affects the data being generated. This means that the data distribution changes during training, introducing instability in the learning process.

RL is highly sensitive to hyperparameter initialization, making training unstable if not tuned correctly. In this project, we analyze two RL algorithms to address the noise and instability issues: **Soft Actor-Critic (SAC)** and **Proximal Policy Optimization (PPO)**.

C. Comparison of PPO and SAC

1) Proximal Policy Optimization (PPO):

- PPO, being an on-policy algorithm, can discard a significant amount of data after each policy update
- Directly optimizes the policy but is **not sample efficient**.
- Sample efficiency is problem-dependent; if data collection is fast (e.g., in a high-speed simulator), sample efficiency is less important, while **computational efficiency** becomes the priority.
- PPO's exploration is primarily driven by noise or clipping, which can be less effective compared to SAC's intrinsic motivation for exploration.

2) *Soft Actor-Critic (SAC)*:

- More **sample efficient** than PPO because it reuses past data for learning.
- Enables **more learning from less data**, making it more suitable for environments where data collection is slow or costly.
- SAC inherently encourages exploration due to its entropy maximization term. This term promotes actions that lead to less predictable outcomes, allowing the agent to discover new and potentially rewarding states.

Based on the given comparison and also training models by both methods, I decided to use SAC. Since the environment is complex and the agent needs more exploration, it seems SAC be more efficient.

3) *How SAC Works (Actor-Critic Framework)*: SAC follows an **off-policy Actor-Critic framework**, where:

- 1) The **current policy (actor)** interacts with the environment to generate actions. The resulting experiences—(*state, action, reward, next state*)—are stored in a **replay buffer**.
- 2) Instead of updating the Q-function immediately, SAC **samples mini-batches** from the replay buffer and applies **gradient descent** to improve both the Q-function and the policy.

Unlike traditional RL methods, SAC **decouples action generation from policy updates**, allowing for **better stability and efficiency** in training by enabling the reuse of past experiences.

D. *Uniform Domain Randomization*

Uniform Domain Randomization (UDR) is implemented by varying the masses of the thigh, leg, and foot across training episodes. This forces the agent to generalize over a distribution of environments. Aims to improve robustness by training the agent on a variety of dynamics, creates a distribution of environments that helps in learning transferable policies, helps prepare the agent for potential variations in the target domain.

E. *Adaptive Domain Randomization (Extention)*

The main concept behind ADR is to automatically adjust the randomization ranges based on the agent's performance, starting with easier variations and gradually increasing difficulty as the agent improves. This is in contrast to UDR which uses fixed randomization ranges throughout training.

IV. EXPERIMENTS

A. *SAC model*

After, evaluating different parameters this model is chosen and all domains are trained on this model.

- **Learning Rate (5e-4)**: Controls the step size in optimization. A moderately high value speeds up learning.
- **Buffer Size (100,000)**: Stores past experiences for training. A reduced size improves efficiency.
- **Batch Size (128)**: Number of samples per training update. A smaller batch size helps maintain stability.

- **Learning Starts (500)**: Training begins after 500 interactions, allowing the agent to gather initial experience before updates.
- **Train Frequency (4, "step")**: The model updates every 4 steps, making updates less frequent for better stability.
- **Gradient Steps (1)**: Number of gradient updates per training step. A conservative update approach prevents instability.
- **Tau (0.01)**: Soft update factor for target networks, ensuring smoother updates to prevent drastic changes.

B. *UDR Performance*

Mass Variation:

Each body part's mass (except the torso) is randomly varied using a uniform distribution with a $\pm 20\%$ range. The base mass of each part is used as the reference for randomization.

Timing:

Randomization of the masses happens at the start of each episode via the 'reset()' function. The randomized mass values stay fixed during each episode. New random values are generated for each new episode.

C. *ADR Performance*

In my implementation, ADR attempts to:

- Starts with conservative variations ($\pm 5\%$).
- Monitors agent performance over episodes.
- Expands ranges when the agent shows consistent success.
- Contracts ranges if the agent struggles.
- Maintains minimum variation to ensure robustness.

1) *Range Adjustment*:

- Tracks performance using rewards and episode lengths.
- Adjusts ranges based on the agent's success rate.
- Uses expansion and contraction rates to modify difficulty.

2) *Adaptation Mechanism*:

- Starts with narrow ranges ($\pm 5\%$ of original masses).
- Expands ranges when the agent performs well ($\text{success_rate} > 0.6$).
- Contracts ranges if the agent struggles.
- Maintains a minimum range width (10%) to prevent over-narrowing.

V. RESULTS AND QUESTIONS

A. *Comparing results*

I did training in source, target, UDR, and ADR domains and tested them on source and target. UDR domain is the source domain which only the torso mass is fixed at 2.53429174 and the other masses are randomized uniformly. ADR domain is the source domain with torso fixed on 2.53429174 but the other masses are randomized adaptively.

My rewards are satisfying but I got several unexpected results on testing the trained models in UDR and ADR domains.

Source Model on Source Domain Results:
Mean Reward: 1180.61 $\pm$ 48.50

Source Model on Target Domain Results:
Mean Reward: 1070.05 $\pm$ 5.64

Target Model on Target Domain Results:
Mean Reward: 1635.82 $\pm$ 9.20

UDR Model on Source Domain Results:
Mean Reward: 1731.05 $\pm$ 5.02

UDR Model on Target Domain Results:
Mean Reward: 1764.76 $\pm$ 5.90

ADR Model on Source Domain Results:
Mean Reward: 1687.36 $\pm$ 5.63

ADR Model on Target Domain Results:
Mean Reward: 1752.47 $\pm$ 34.90

Fig. 1. Evaluation results

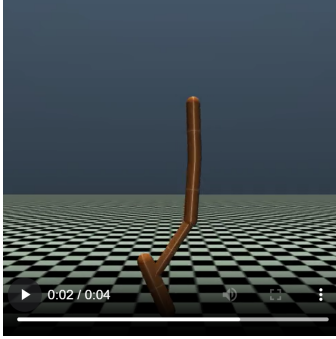


Fig. 2. Simulation of hopper trained with ADR

B. Question 1.1

The Hopper environment provides an **11-dimensional continuous state representation**, including:

- **Positional Information:**
 - Hopper height (`rootz`)
 - Angles of torso and joints (`thigh`, `leg`, `foot`)
- **Velocity Information:**
 - Horizontal and vertical velocity of the torso.
 - Angular velocities of all joints.

These observations allow the agent to learn a dynamic control policy for stable hopping.

C. Question 1.2

The action space is **continuous**, represented as `Box(-1, 1, (3, ), float32)`. The agent controls three joints:

- **Thigh Joint** – Controls vertical stability.
- **Leg Joint** – Affects propulsion and landing impact.
- **Foot Joint** – Assists in maintaining balance.

The torque values range from **-1 to 1**, normalized and scaled to environment-specific limits.

D. Question 1.3

In source and target environment the mass of thigh = 3.92699082, leg = 2.71433605, foot = 5.0893801, but torso in target is 3.53429174 while in source is 2.53429174.

E. Question 3.1

We expect the performance of trained model in source domain on the target domain be less than source it self, since the agent has learned the policy on source and the change in target domain introduce new challenges for agent.

F. Question 3.2

Training directly on the **target (real-world) environment** is impractical due to the following reasons:

- **Safety Risks** Unstable policies can damage the robot or cause unsafe behaviors.
- **High Costs** Real-world data collection is expensive and resource-intensive.
- **Time Constraints** RL requires millions of interactions, which is impractical in real time.
- **Reset Challenges** Unlike simulation, real-world resets take manual effort.
- **Reproducibility Issues** Real-world conditions introduce variability (e.g., sensor noise, wear and tear).

G. Question 4.1

Yes actually it introduces a lot of improvements in rewards and the agent moves more stabilized. It also reduces performance degradation when transitioning from simulation to real-world conditions.

H. Question 4.2

The agent may learn overly cautious behaviors, reducing optimal performance. Randomizing too many parameters increases training time and sample complexity. Limited Realism UDR assumes a uniform distribution of variations, which may not reflect real-world uncertainties accurately. Poor Adaptation to Outliers The policy might struggle with unseen extreme cases that were not well covered in training.

VI. DISCUSSION

UDR reduces the performance gap between the source and target environments. However, its effectiveness depends on the choice of randomized parameters. More advanced methods could explore Adaptive Domain Randomization (ADR) for more targeted generalization improvements but it needs accurate parameters.

A. Modification on ADR

- The performance history window length plays a crucial role in determining how quickly ADR adapts to new challenges. The current implementation uses a 100-episode window, while a 200-300 episode window could provide more stable statistics for better decision-making about when to expand or contract the randomization ranges.
- The mass range adaptation thresholds significantly impact exploration versus exploitation. The current thresholds (expansion at 200, contraction at 100) could be adjusted to create a more nuanced adaptation scheme. Using an intermediate threshold around 150 would help maintain

stability when performance is adequate but not exceptional.

- The granularity of mass range adjustments affects how smoothly the system adapts. The current step size of 0.05 for all body parts could be refined to use smaller steps (0.02-0.03) for more precise control. Different step sizes could be used for different body parts based on their sensitivity to mass changes.
- Body-part specific randomization ranges could better reflect the different roles of each component. The thigh, being larger, could safely handle wider mass variations (0.7-1.3) compared to the foot, which might benefit from a narrower range (0.8-1.2) to maintain stability.

VII. CONCLUSION

This project demonstrates that reinforcement learning can be applied to robotic control with sim-to-real transfer improvements using domain randomization. More improvement could be achieved with adaptive domain randomization.

VIII. IMPLEMENTATION

Uploaded files consist of custom-hopper with the UDR and ADR code, one policy file of colab code, a train and test python files, report pdf along with 4 trained models. I had a lot of problems with installations and for simulation I use colab.

REFERENCES

- [1] R. Sutton and A. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 1998.
- [2] J. Tobin et al., "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," arXiv:1703.06907, 2017.
- [3] X. B. Peng et al., "Sim-to-real transfer of robotic control with dynamics randomization," *IEEE International Conference on Robotics and Automation*, 2018.